

Mock 1 — Gajanan (Architect)

20 minutes · timed script · answer from memory

Mock interview

20 min

Architecture

Tone to expect: collaborative, not adversarial. He is 24 years senior — he does not need to prove anything. Danger is not hostility, it is that you will say a buzzword (“multimodal”, “multi-objective”, “HistGradientBoosting”) and he will quietly ask you to define it. Every noun on your resume is a door he can open.

How to use. Set a 20-minute timer. Read Gajanan’s line aloud (or have someone read it). Answer the green box **from memory**, out loud, then compare. Do not read ahead.

Intro to memorise (35–40 sec):

Good morning sir, I am Kavya Bhatiya, B.Tech in Artificial Intelligence at SVNIT Surat, CGPA 9.05. My main project is LogiFlow — a multimodal AI logistics platform on FastAPI, PostgreSQL and Next.js, with Gemini and a HistGradientBoosting delay-risk service — 73 percent accuracy, 68 percent delayed-shipment recall, 78.3 percent ROC-AUC. It reached Top 106 globally in the Google Solution Challenge 2026. I also do research on multi-objective recommenders using NSGA-II in SMSRF — up to 12.6 percent higher novelty and 20.9 percent higher serenity than an NSGA-3 baseline. I write C, C++ and Python, and I have solved 470-plus problems on LeetCode at a 1620 rating.

Three rules for this round. (1) Never say a number you cannot break down — he *will* ask how it was measured. (2) When you name a technology, immediately give the alternative you rejected. (3) If you do not know: “I have not built that; the closest thing I have done is ____.” Say it flatly. Architects respect a clean boundary far more than a bluffed one.

Clock

0:00–2:00	Open	Intro, why AI branch, one line on Vadodara/SVNIT if asked.
2:00–7:00	LogiFlow: product then boxes	What problem, who uses it, then the component diagram in words.
7:00–11:00	Docker / reliability / secrets	Why Docker, what is in the image, secrets, failure modes.
11:00–15:00	Delay model honesty	73% accuracy, 0.68 recall, 0.783 ROC-AUC — defend, do not oversell.
15:00–17:30	Second artefact	SMSRF NSGA-II (preferred) or AirHelp / MindSettler.
17:30–19:00	Light OOP / design	Abstraction, strategy pattern, or “how would you extend it”.
19:00–20:00	Close	Your question for him + thank you.

Script

0:00 — Opening

INTERVIEWER Good morning Kavya. I am Gajanan, I work as a software architect here at GE HealthCare in Bengaluru. Before we start — tell me about yourself, and keep it to about a minute.

YOU SAY

YOU SAY

The 35–40 second intro above. Then **stop talking**. Do not fill silence.

INTERVIEWER *You are in the AI branch — that is still fairly new at NITs. Was that your first choice, or did you land there?*

YOU SAY

YOU SAY

It was a deliberate choice, sir. I wanted the systems fundamentals — DSA, OS, computer networks, DBMS — but with the modelling layer on top, because most of what I build ends up being an ordinary backend that happens to have a model inside it. LogiFlow is exactly that: ninety percent API, database and deployment work, ten percent model.

2:00 — LogiFlow: product first, then architecture

INTERVIEWER *Let us take LogiFlow. Assume I know nothing about logistics. What is the problem, who is the user, and what do they get out of it?*

YOU SAY

YOU SAY

The user is someone who has to move freight between two cities and has more than one way to do it — road, rail, or a combination. Today that decision is made on phone calls and habit. LogiFlow takes the origin, destination, and the user's constraints — budget, delivery deadline, how much delay risk they can tolerate — and returns a ranked set of route options where each one is scored on cost, transit time, and probability of delay. The deliverable is not a map, sir, it is the trade-off: this route is cheaper but eighteen percent likelier to be late, this one costs more and is reliable. The user picks; the system makes the picking informed.

INTERVIEWER *Good. Now draw me the architecture — in words. What are the boxes, and what talks to what?*

YOU SAY

YOU SAY

Four layers. Front-end is Next.js — route comparison UI, no business logic. It talks over HTTPS to a FastAPI service, which is the only thing that touches data. Inside FastAPI: validation/auth, routing and multi-objective scoring (delivery time, reliability, cost, sustainability), and an inference module that loads the HistGradientBoosting delay-risk model. Persistence is PostgreSQL — corridors, legs, tariffs, history, planner data. Gemini is a separate outbound call for document/field assistance and explanation text — convenience path, not the ranking path. The backend is packaged with Docker (on my resume Tools line); the team hosts the containerised API in production.

INTERVIEWER *Why is Gemini a separate box in your description? What does that tell you about it?*

YOU SAY

YOU SAY

Because it is the only dependency I do not control, sir. It is a third-party call over the network with variable latency and a quota, so architecturally I treat it as untrusted and optional: it runs on the ingestion path, not on the ranking path. If Gemini is slow or down, the user can still type the shipment details manually and get a full ranked answer. The extraction is a convenience layer, so a failure there degrades the experience but does not take the product down.

INTERVIEWER *And your ranking — you have written “multi-objective ranking”. Define that for me. What exactly is multi-objective about it?*

YOU SAY

YOU SAY

Three objectives that genuinely conflict: minimise cost, minimise transit time, and minimise delay probability. The cheapest route is usually rail and usually slow; the fastest is usually road and usually costlier. Because there is no single best answer, I do not collapse them into one number and hide it. I normalise each objective onto a common scale and expose weights the user can shift — cost-sensitive versus deadline-sensitive — and the ranking recomputes. When I need a defensible set rather than a weighted sum, I look for the non-dominated options, meaning a route stays in the shortlist only if nothing else beats it on all three at once.

INTERVIEWER *So you are essentially doing Pareto filtering. Where else have you used that?*

YOU SAY

YOU SAY

In my research work, sir — SMSRF, where I used NSGA-II, a genetic algorithm, to optimise a recommender system across accuracy and novelty simultaneously. That is where the habit came from. LogiFlow uses the idea, not the evolutionary machinery — for three objectives and a bounded candidate set, running a genetic algorithm per request would be an unjustifiable latency cost. Direct scoring with dominance filtering gives the same shortlist in milliseconds.

7:00 — Docker, reliability, secrets

INTERVIEWER *Your resume lists Docker. What does containerising LogiFlow actually buy you?*

YOU SAY

YOU SAY

Same artefact locally and in production, sir. Slim Python image, dependencies locked, FastAPI starts uvicorn on the configured port, model artefact loaded once at process start. Secrets stay out of the image — env or a secret store at runtime, gitignored dotenv locally. I do not claim I personally operated every production host knob; I own the Docker packaging and the service behaviour inside the container.

INTERVIEWER Fair. Suppose traffic spikes and you run many container copies against Postgres. What goes wrong?

YOU SAY

YOU SAY

Connection exhaustion. Each instance opens a pool; N times pool size can exceed Postgres max connections under the load you wanted to serve. Keep pools small, cap instance count, and in production put a pooler in front. State stays in Postgres — any instance can serve any request.

INTERVIEWER And credentials — database password, Gemini key — where do they live?

YOU SAY

YOU SAY

Not in the image and not in the repository. Injected at runtime. Locally dotenv is gitignored. If a key might have been logged, rotate it.

Gajanan. Suppose tomorrow this has to run inside a hospital network with no internet egress. What breaks?

YOU SAY

YOU SAY

Gemini breaks first — outbound SaaS. Because it sits behind an interface on the ingestion/explain path, I swap to local extraction or manual entry and ranking stays up. FastAPI and Postgres are fine on-prem; same Docker image, different orchestrator. The delay model must be retrained on in-network data — I do not assume public logistics features transfer into a clinical setting.

11:00 — The delay model: honesty block

INTERVIEWER Your resume says the delay model gives 73 percent accuracy, 68 percent recall on delayed shipments, and 78.3 ROC-AUC. Talk me through those three numbers. Why did you print all three?

YOU SAY

YOU SAY

Because accuracy alone would be misleading, sir, and I did not want to hide behind it. The classes are imbalanced — most shipments arrive on time — so a model that predicts “on time” for everything scores respectably on accuracy while being useless for the only case the user cares about. So 73 percent accuracy is the honest headline, 68 percent recall on the delayed class is the number that says I actually catch roughly two-thirds of real delays, and 0.783 ROC-AUC is the threshold-independent measure showing the model separates the classes meaningfully better than chance, which is 0.5. Read together, they say: useful signal, not a solved problem.

Gajanan. Two-thirds. So one delayed shipment in three, you miss. Would you ship that?

YOU SAY

YOU SAY

I would ship it as an advisory, not as a guarantee — and that is exactly how it is surfaced in the product. It is presented as a delay risk score contributing to a ranking, never as “this shipment will be late”. The user is still choosing between routes, so a directionally correct risk ordering already improves the decision over having no signal. I would not ship this same model into a workflow that automatically cancelled or rebooked a shipment; that needs much higher precision and a human confirmation step.

INTERVIEWER *You said threshold-independent. So you can move the threshold. Which way would you move it, and what does it cost you?*

YOU SAY

YOU SAY

Lower it, because the errors are not symmetric. A false positive means I warned about a delay that did not happen — the user was mildly over-cautious. A false negative means the shipment was late and I said nothing, and that is the expensive one. So I would lower the decision threshold to buy recall, and I pay for it in precision with more false alarms. Where exactly to stop is not a modelling question, it is a question for whoever owns the operational cost — so I would ask for the ratio of what a missed delay costs versus a false alarm and set the threshold from that.

INTERVIEWER *Which model, and did you try others?*

YOU SAY

YOU SAY

The production model is histogram-based gradient boosting. I evaluated CatBoost as well, and I use CatBoost elsewhere. Gradient boosting on trees won because the features are tabular and mixed — distances, mode, corridor identity, seasonality — where trees handle non-linear interactions without me hand-crafting them, and histogram binning made training fast enough to iterate in minutes. I also kept a logistic regression baseline, which matters because without it I could not claim the boosting was earning its complexity. And I checked one thing specifically: that I had no feature which leaks the outcome — nothing that would only be knowable after the shipment had already arrived late.

INTERVIEWER *How did you split your data?*

YOU SAY

YOU SAY

The honest answer is that the split is the weakest part of this project. Delay data is temporal, so a random split lets the model see future shipments while predicting past ones, which flatters the metrics. The defensible approach is a time-based split — train on earlier periods, validate on the most recent — and if I were redoing this that is the first change I would make, and I would expect the numbers to come down somewhat and be more trustworthy. I would rather tell you that than quote a number I cannot stand behind.

15:00 — Second artefact (he will pick one)

INTERVIEWER *Tell me briefly about the research — SMSRF, NSGA-II. What did you actually do, and what is the result?*

YOU SAY

YOU SAY

Recommender systems are usually optimised for accuracy alone, which collapses recommendations into popular items — the user sees what everybody sees. I formulated it as a multi-objective problem and used NSGA-II, an evolutionary algorithm that maintains a population of candidate solutions and evolves a Pareto front instead of one optimum. Evaluated on MovieLens, against an NSGA-3 baseline we improved novelty by 12.6 percent and serenity by 20.9 percent. The engineering lesson I carried out of it is that when objectives conflict, publishing the front and letting the operator choose the operating point is more honest than shipping one hard-coded weighting.

INTERVIEWER *Twenty percent on a metric called serenity — is that a metric a user would feel?*

YOU SAY

YOU SAY

Not directly, sir. It is a proxy for how comfortable a recommendation set is for the user rather than how accurate it is, and I would not claim a user perceives a twenty percent improvement. What is defensible is the comparison: same dataset, same protocol, same baseline, so the relative gain is real even if the absolute human impact is not something I measured. A user study is what would be needed for that claim, and we did not run one.

INTERVIEWER *And AirHelp — one line. What is different there from LogiFlow?*

YOU SAY

YOU SAY

AirHelp is an airport assistance assistant — FastAPI backend, Next.js front-end, with Gemma as the language model for handling passenger queries. Architecturally the difference is that in AirHelp the model is the product, so latency and prompt handling are the whole engineering problem, whereas in LogiFlow the model is one component inside a decision system. MindSettler is my third one, a counselling appointment platform on Django REST with SQL — that one was where I actually learned authentication and relational schema design properly, before I moved to FastAPI.

17:30 — Light OOP / design (only if time survives)

INTERVIEWER *Small design question. Tomorrow the product team says LogiFlow must support air freight as a fourth mode. What do you change?*

YOU SAY

YOU SAY

Ideally very little in the ranking layer, and that was the intent of the design. Each transport mode sits behind a common interface — given an origin, destination and constraints, return candidate legs with cost, time and features. Rail, road and multimodal are implementations of that. Air freight becomes a fourth implementation plus its own data, and the scorer and ranker do not change because they consume the interface, not the concrete mode. Realistically, two things would still break: the delay model has no air-freight training data, so its predictions would be out of distribution and I would gate air routes to a heuristic until I had data; and air has constraints the schema does not model, like cargo weight limits, so there is a migration.

INTERVIEWER *You just described a pattern without naming it. What is it, and why not just use an if-else on mode?*

YOU SAY

YOU SAY

That is the strategy pattern, sir — behaviour selected at runtime behind a shared abstraction. The reason not to use if-else on mode is that the conditional does not stay in one place. It starts in the router, then appears in pricing, then in scoring, and adding a mode means finding every branch and hoping you found them all. With the interface, adding a mode is one new class registered in one factory, and the existing paths are not touched — which means the existing tests still mean something. It is the open-closed principle in practice: open to extension, closed to modification.

INTERVIEWER *One fundamentals question. Why would you make a method virtual in C++?*

YOU SAY

YOU SAY

So the call resolves on the actual object's type at runtime rather than the pointer's declared type. Without virtual, calling through a base pointer runs the base version — static binding — which defeats the purpose of the hierarchy. With virtual, dispatch goes through the vtable to the derived override. Two related points I keep in mind: a base class with virtual functions needs a virtual destructor, otherwise deleting a derived object through a base pointer only runs the base destructor and leaks; and virtual dispatch costs an indirection, so it is not free — in a tight numerical loop I would avoid it.

19:00 — Close

INTERVIEWER *We are nearly out of time. Do you have questions for me?*

YOU SAY

YOU SAY

Two if there is time, sir. First — given your architecture role, how much of an Edison intern’s work is inside an existing service versus greenfield? I want to know whether I should be preparing to read a large existing codebase, because that is a skill I would like to be tested on. And second, healthcare software has constraints I have never worked under — validation, regulatory, on-premise hospital deployment. How early does an intern encounter those, and is there something you would recommend I read before joining?

INTERVIEWER Good questions. Thank you Kavya, we will get back to you through the campus process.

YOU SAY

YOU SAY

Thank you sir. I enjoyed the conversation — particularly the point on air-gapped deployment; that is something I had not thought through before.

Score this mock

Rate each area /10. Pass bar: 7+ average, and nothing below 5.

Intro	Under 45 s? Named LogiFlow with one proof and one number? Stopped cleanly?	
Product framing	Explained LogiFlow with zero framework names? User + problem + outcome?	
Architecture	Named the boxes and what talks to what? Separated critical path from convenience path?	
Trade-offs	Every technology came with the rejected alternative — FastAPI vs Django, HistGB vs baseline, Postgres vs Mongo?	
Reliability	Docker image, secrets, connection limits, no-egress scenario — answered without panic?	
Metric honesty	Explained <i>why</i> three metrics? Owned the temporal-split weakness with a fix?	
Design / OOP	Strategy pattern reasoning? Virtual + virtual destructor without prompting?	
Close	Question aimed at his world, not at perks?	

Pass bar for tomorrow. You never said “multi-objective” or “multimodal” without immediately defining it in one plain sentence. You quoted 73 / 0.68 / 0.783 and could explain why each exists. You said “I have not built that” at least once without flinching. You did not claim the delay model was production-grade.

Three killers to avoid. (1) Reciting the stack as a list — he does not care that it is FastAPI, he cares why not Django. (2) Saying “accuracy is 73 percent” and stopping. (3) Inflating the research — it is a strong undergraduate contribution, do not call it state of the art.

Next: Kavya_GEHC_Mock2_Classic_Campus.pdf — same timer, C++ OOP and live coding, no notes.